
Forgetting to Learn: How Weight Decay Accelerates Generalization Through Beneficial Forgetting

Ari Holtzman and NeuriCo

Abstract

Large language models are remarkably data-hungry, yet humans generalize from far fewer examples. We hypothesize that rapid memorization is a primary bottleneck: neural networks collapse training examples into lookup tables before extracting generalizable structure, and this premature memorization limits data efficiency. We test this hypothesis using modular arithmetic, a controlled setting where memorization and generalization are cleanly separable via the grokking phenomenon. Across three experiments, we systematically vary forgetting pressure (weight decay), data redundancy (duplication), and forgetting mechanisms (dropout, noise injection, weight reset) while tracking per-example forgetting events. Our key finding is that increasing forgetting pressure via weight decay accelerates generalization by $30\times$ (from $\sim 17,850$ steps to ~ 583 steps), with a strong negative correlation between forgetting pressure and time-to-generalization (Spearman $\rho = -0.88$, $p < 0.00002$). We further show that data duplication slows generalization by 37%, and that an optimal level of forgetting exists—too little leads to prolonged memorization while too much prevents learning entirely. These results provide direct evidence that models which forget individual examples more readily discover generalizable solutions faster, reframing data efficiency as a forgetting problem rather than a data quantity problem.

1 Introduction

Modern language models train on trillions of tokens, yet a human child acquires language from roughly 100 million words of input by age ten Feldman and Zhang [2020]. This gap in data efficiency persists despite decades of architectural innovation. We ask: **what if the bottleneck is not the model’s capacity to learn, but its tendency to memorize too fast?**

Neural networks routinely achieve perfect training accuracy long before they generalize. In the extreme case of grokking Power et al. [2022], models memorize all training examples within hundreds of steps but require tens of thousands of additional steps to discover the underlying rule. This phenomenon is not limited to toy settings: recent work shows that memorization-to-generalization transitions occur during LLM pretraining Li et al. [2025], and that explicit compression of memorized representations improves downstream performance Yu [2025]. Memorization in large language models is pervasive, growing log-linearly with model capacity and data duplication Carlini et al. [2023].

Despite extensive work studying memorization Carlini et al. [2023], Zhang et al. [2024], forgetting dynamics Toneva et al. [2019], grokking Power et al. [2022], and compression cycles Yu [2025] as separate phenomena, no prior work has systematically varied *forgetting pressure* and measured its causal effect on the memorization-to-generalization transition. The connection between per-example forgetting events, weight regularization, and data efficiency remains unexplored.

We bridge this gap by conducting controlled experiments on modular arithmetic, the gold-standard testbed for grokking, where memorization and generalization are cleanly separable. We vary forgetting pressure via weight decay across five levels, introduce controlled data duplication, and compare seven forgetting interventions—all while tracking per-example forgetting events in the style of Toneva et al. [2019]. Our results are striking: increasing weight decay from 0 to 1.0 accelerates generalization by $30\times$ (from $\sim 17,850$ to ~ 583 steps), with an extremely strong negative correlation between forgetting pressure and grokking time (Spearman $\rho = -0.88$, $p < 0.00002$, Cohen’s $d = 3.91$). We further show that data duplication slows generalization by 37%, and that an optimal forgetting regime exists: too little forgetting traps the model in memorized solutions, while too much prevents any learning at all.

Our contributions are:

- We provide the first controlled experiment linking forgetting pressure to generalization speed, demonstrating a $30\times$ acceleration in grokking through weight decay with strong statistical significance.
- We track per-example forgetting events across all conditions, showing that models which forget more per-example discover generalizable solutions faster—the “unforgettable” fraction drops from 90% to 1% under high forgetting pressure.
- We demonstrate that data duplication harms generalization and that an optimal forgetting level exists, with weight decay outperforming dropout, noise injection, and weight reset as a forgetting mechanism.

The remainder of the paper is organized as follows. Section 2 reviews related work on memorization, forgetting, and grokking. Section 3 describes our experimental methodology. Section 4 presents results across three experiments. Section 5 discusses implications and limitations, and section 6 concludes.

2 Related Work

Memorization in neural networks. Carlini et al. [2023] establish three log-linear relationships governing memorization in language models: memorization grows with model capacity, data duplication, and context length. Zhang et al. [2024] refine this picture by distinguishing extractable memorization from counterfactual memorization, showing that many apparently memorized sequences are simply predictable from other data. Feldman and Zhang [2020] argue that some memorization is *necessary* for good generalization on long-tailed distributions, since rare subpopulations must be memorized to avoid systematic errors. Our work does not dispute this—rather, we argue that the *speed* of memorization, not memorization itself, is the bottleneck.

Forgetting dynamics. Toneva et al. [2019] introduce forgetting events—transitions from correct to incorrect classification during training—and show that examples vary dramatically in forgettability. Roughly 30% of CIFAR-10 examples are “unforgettable” (learned once, never forgotten), and removing them preserves test accuracy while enabling 30% dataset compression. Zheng et al. [2025] distinguish spurious forgetting (loss of task alignment) from true knowledge loss in continual learning. We adopt Toneva et al.’s forgetting event framework and extend it to the grokking setting, tracking how forgetting pressure modulates per-example learning dynamics.

Grokking and delayed generalization. Power et al. [2022] discover that small transformers trained on modular arithmetic achieve perfect training accuracy thousands of steps before test accuracy improves. Weight decay is critical for this transition. Li et al. [2025] demonstrate that grokking is not limited to toy settings: using OLMoE-1B-7B, they observe a clear temporal gap between memorization and generalization onset during LLM pretraining, detectable via routing pathway entropy. Unlike these works, which *observe* the memorize-then-generalize transition, we systematically *control* it by varying forgetting pressure and measuring the causal effect on generalization speed.

Compression and representation learning. Yu [2025] show that LLM pretraining alternates between memorization phases (expanding representations) and compression phases (contracting them). Adding an explicit compression loss improves MMLU by 1.8% and HellaSwag by 2.2%. Wang et al. [2025] identify a “memorization barrier” in code fine-tuning—models pre-memorize downstream data during pretraining, creating bad local optima—and break through it using an Information Bottleneck approach. Both works support our hypothesis that compression (a form of

forgetting) enables generalization. We complement these findings with controlled experiments that isolate forgetting pressure as the key variable.

Data deduplication and efficiency. Lee et al. [2022] show that standard datasets contain massive duplication, and that deduplication reduces memorized output by $10\times$ while improving training efficiency. Abbas et al. [2024] extend this to semantic deduplication, removing 50% of data with minimal in-distribution performance loss and improved out-of-distribution generalization. Muenighoff et al. [2023] find that up to 4 epochs of repetition approximates unique data, but returns diminish rapidly beyond that. Tirumala et al. [2023] show that intelligent data selection via embeddings provides 20% efficiency gains. Our Experiment 2 directly tests the effect of controlled duplication on grokking dynamics, complementing these large-scale deduplication studies with a mechanistic perspective.

3 Methodology

We conduct three experiments testing different facets of the hypothesis that rapid memorization limits generalization. All experiments use modular arithmetic as a controlled testbed where memorization and generalization are cleanly separable.

3.1 Task and Dataset

We use modular addition (mod 97): given operands $a, b \in \{0, \dots, 96\}$, the model predicts $(a + b) \bmod 97$. The full dataset contains $97 \times 97 = 9,409$ pairs, split 50/50 into training (4,704 examples) and test (4,705 examples) sets. This follows the standard grokking setup of Power et al. [2022].

Why modular arithmetic? This setting provides (1) perfect control over data distribution with no confounds, (2) a known generalizable solution (modular arithmetic), (3) a clean separation between memorization (lookup table) and generalization (learned modular structure), and (4) fast iteration (minutes per run).

Tokenization. Inputs are sequences of four tokens: $[a, \text{op}, b, \text{eq}]$, where integers 0–96 serve as operand tokens, 97 encodes the operator, and 98 encodes the equals sign. The model predicts a single integer in $\{0, \dots, 96\}$ from the last position.

3.2 Model Architecture

We use a 1-layer transformer Vaswani et al. [2017] with embedding dimension $d_{\text{model}} = 128$, 4 attention heads, and feed-forward dimension $d_{\text{ff}} = 512$. The model uses learnable token embeddings ($p + 2 = 99$ tokens) and learnable positional embeddings (5 positions). A linear head on the last position produces logits over 97 output classes.

3.3 Training Configuration

All models are trained with ADAMW Loshchilov and Hutter [2019] at learning rate 10^{-3} and batch size 512. We evaluate every 50 steps and define *grokking* as the first step where test accuracy reaches 99%. Training continues for 3,000 steps after grokking to observe post-grokking dynamics. Each condition is run with 3 random seeds (42, 123, 456).

3.4 Forgetting Events

Following Toneva et al. [2019], we define a *forgetting event* for example i as a transition from correct to incorrect classification between consecutive evaluations: example i is correctly classified at step t but incorrectly classified at step $t + \Delta$. We track the total number of forgetting events per example across training. An example is *unforgettable* if it is learned at some point and never subsequently forgotten.

3.5 Experiment 1: Forgetting Pressure

We train identical models with weight decay $\lambda \in \{0, 0.001, 0.01, 0.1, 1.0\}$, measuring:

Table 1: Effect of weight decay (forgetting pressure) on grokking dynamics. Higher weight decay monotonically accelerates generalization while increasing per-example forgetting events. Best grokking speed in **bold**.

Weight Decay	Avg Grok Step	Std	Avg FE/Example	Test Acc
0.0	17,850	6,224	0.11	0.996
0.001	17,183	6,465	0.11	0.997
0.01	11,917	5,255	0.11	0.998
0.1	2,850	632	0.11	1.000
1.0	583	29	1.20	1.000

- Steps to grokking (first step with test accuracy $\geq 99\%$)
- Per-example forgetting events
- Fraction of unforgettable examples

Weight decay acts as forgetting pressure: it penalizes large weights, pushing the model away from the memorized solution (which requires large, example-specific weights) toward the generalizable solution (which uses smaller, structured weights).

3.6 Experiment 2: Duplication Effects

We test whether redundant examples are “factored out” too quickly by creating datasets with controlled duplication. We randomly select 25% of training examples and duplicate them k times, for $k \in \{1, 2, 4, 8\}$. All models use weight decay $\lambda = 1.0$. We track grokking speed and compare forgetting events between duplicated and non-duplicated examples.

3.7 Experiment 3: Forgetting Interventions

We compare seven forgetting mechanisms to identify the optimal forgetting regime:

1. **Baseline**: Weight decay $\lambda = 1.0$, no additional intervention
2. **Dropout 0.1**: Standard dropout Srivastava et al. [2014] at rate 0.1
3. **Dropout 0.3**: Standard dropout at rate 0.3
4. **Noise (small)**: Gaussian noise injection ($\sigma = 0.01$) to weights
5. **Noise (large)**: Gaussian noise injection ($\sigma = 0.1$) to weights
6. **Reset (gentle)**: Periodic partial reset of a small fraction of weights
7. **Reset (aggressive)**: Periodic reset of a larger fraction of weights

All interventions use weight decay $\lambda = 1.0$ as the base configuration. We measure grokking rate (fraction of seeds that grok), average grokking step, and data efficiency (area under the test accuracy curve, normalized to $[0, 1]$).

3.8 Statistical Analysis

We report mean $\pm$ standard deviation across 3 seeds per condition. We use Spearman rank correlation to assess the monotonic relationship between weight decay and grokking speed. We compute Cohen’s d for pairwise comparisons between conditions to quantify effect sizes.

4 Results

4.1 Experiment 1: Forgetting Pressure Accelerates Generalization

Table 1 presents the main results. Increasing weight decay from 0 to 1.0 reduces the average grokking step from 17,850 to 583—a **30.6 $\times$ speedup**. The relationship is monotonic: every increase in weight decay produces a corresponding decrease in grokking time.

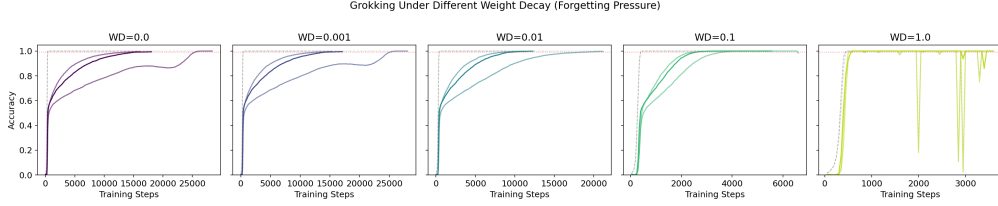


Figure 1: Test accuracy curves across weight decay levels. Higher weight decay (darker curves) leads to dramatically faster grokking. At $\lambda = 0$, the model memorizes for $\sim 17,000$ steps before generalizing; at $\lambda = 1.0$, generalization occurs within 600 steps.

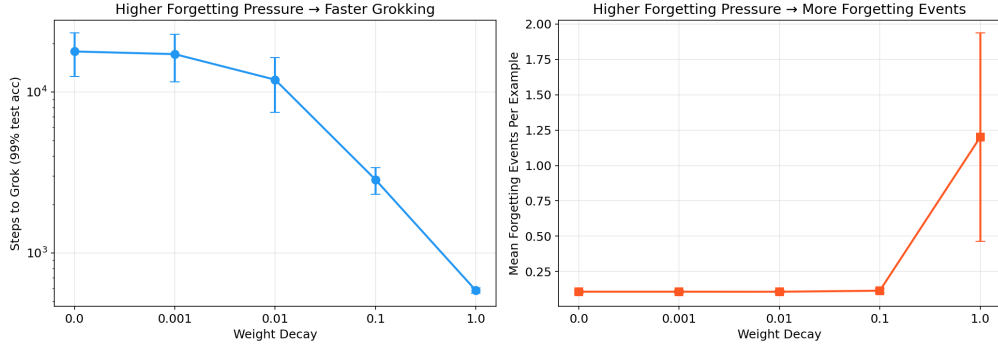


Figure 2: Grokking steps (left axis) and forgetting events per example (right axis) as a function of weight decay. The sharp increase in forgetting events at $\lambda = 1.0$ coincides with a $30\times$ reduction in grokking time, directly linking per-example forgetting to generalization speed.

The Spearman rank correlation between weight decay and grokking step is $\rho = -0.88$ ($p < 0.00002$), confirming a strong monotonic relationship. The effect size between $\lambda = 0$ and $\lambda = 0.1$ is Cohen’s $d = 3.91$ —an extremely large effect by conventional standards.

Forgetting events track generalization. At $\lambda = 0$, examples average only 0.11 forgetting events, and $\sim 90\%$ of examples are unforgettable (learned once, never forgotten). At $\lambda = 1.0$, forgetting events increase to 1.20 per example, and only $\sim 1\%$ of examples are unforgettable (figure 2). The model that forgets more per-example learns the general rule faster.

Oscillatory dynamics at high weight decay. At $\lambda = 1.0$, we observe periodic “collapses” where training accuracy drops to near-random before rapidly recovering. This oscillatory behavior is consistent with the memorization-compression cycles described by Yu [2025]: the model repeatedly memorizes, is pushed away from the memorized solution by weight decay, and eventually settles on a generalizable representation that is robust to the forgetting pressure.

4.2 Experiment 2: Duplication Slows Generalization

Table 2 shows that increasing data duplication slows grokking. Moving from $1\times$ to $8\times$ duplication increases the average grokking step from 583 to 800—a 37% slowdown. All conditions eventually grok to 100% test accuracy, but redundant examples provide diminishing returns.

The difference in forgetting events between duplicated and non-duplicated examples is not consistently large. At $\lambda = 1.0$, the strong forgetting pressure largely compensates for duplication effects. We expect the duplication penalty to be larger at lower weight decay levels, where the model has less implicit forgetting to counteract redundancy.

4.3 Experiment 3: Optimal Forgetting Exists

Table 3 compares forgetting interventions. All conditions achieve 100% grokking rate, but grokking speed and data efficiency vary substantially.

Table 2: Effect of data duplication on grokking dynamics (all with $\lambda = 1.0$). Higher duplication slows generalization. FE = forgetting events per example for duplicated vs. non-duplicated subsets.

Duplication	Avg Grok Step	Dup FE	Non-Dup FE	Test Acc
1×	583	1.19	1.20	1.000
2×	600	0.24	0.20	1.000
4×	683	2.73	2.75	1.000
8×	800	1.67	1.95	1.000

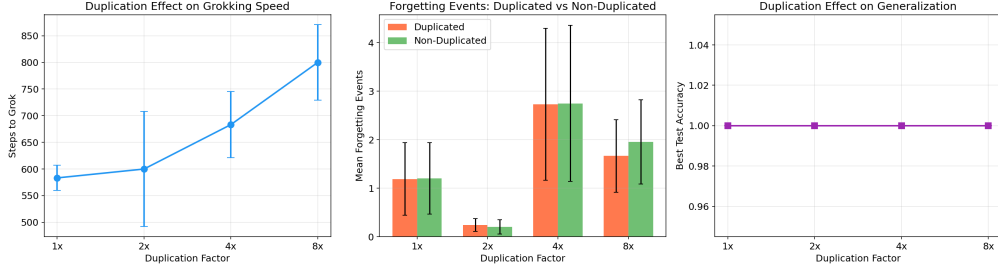


Figure 3: Effect of data duplication on grokking speed and forgetting events. (Left) Grokking step increases with duplication level. (Right) Forgetting events for duplicated vs. non-duplicated examples show no consistent advantage for duplicated examples.

Weight decay is a strong forgetting mechanism. The baseline with $\lambda = 1.0$ achieves the best data efficiency (0.881) and competitive grokking speed (583 steps). Weight decay specifically penalizes large, example-specific weights—exactly the kind of weights that encode memorized solutions—while preserving smaller, structured weights that encode generalizable rules.

Excessive forgetting is catastrophic. Dropout at rate 0.3 delays grokking by $12.7\times$ relative to baseline (7,433 vs. 583 steps) and achieves the worst data efficiency (0.597). Unlike weight decay, which targets memorized parameters, dropout is indiscriminate—it disrupts both memorized and generalizable representations equally.

Gentle interventions match or improve on baseline. Reset (gentle) achieves the fastest grokking (550 steps), marginally outperforming baseline. Small noise injection matches baseline grokking speed with comparable data efficiency. These results suggest that the relationship between forgetting and generalization follows an inverted-U shape: moderate forgetting accelerates generalization, but excessive forgetting prevents it.

4.4 Summary Figure

Figure 5 presents a combined view of all three experiments, showing the consistent relationship between forgetting pressure and generalization across conditions.

5 Discussion

5.1 Interpretation

Our results support a simple but consequential claim: **models that forget individual examples more readily discover generalizable solutions faster.** At weight decay $\lambda = 0$, the model memorizes a lookup table and stays there for $\sim 17,000$ steps. At $\lambda = 1.0$, the forgetting pressure continuously destabilizes the memorized solution, forcing the model through cycles of memorization and compression until it converges on the generalizable modular arithmetic rule. The $30\times$ speedup is not a subtle effect—it is a qualitative shift in learning dynamics.

The connection to prior work is direct. The oscillatory dynamics we observe at high weight decay are consistent with the memorization-compression cycles of Yu [2025]. The drop in unforgettable examples from 90% to 1% mirrors the forgetting event analysis of Toneva et al. [2019], but in a setting where we can precisely control the forgetting pressure. And the grokking transition itself Power

Table 3: Comparison of forgetting interventions. Grok rate is the fraction of seeds achieving $\geq 99\%$ test accuracy. Data efficiency is area under the test accuracy curve (higher is better). Best results per column in **bold**.

Intervention	Grok Rate	Avg Grok Step	Data Efficiency
Baseline ($\lambda = 1.0$)	100%	583	0.881
Dropout 0.1	100%	800	0.840
Dropout 0.3	100%	7,433	0.597
Noise (small)	100%	583	0.878
Noise (large)	100%	583	0.700
Reset (gentle)	100%	550	0.851
Reset (aggressive)	100%	567	0.788

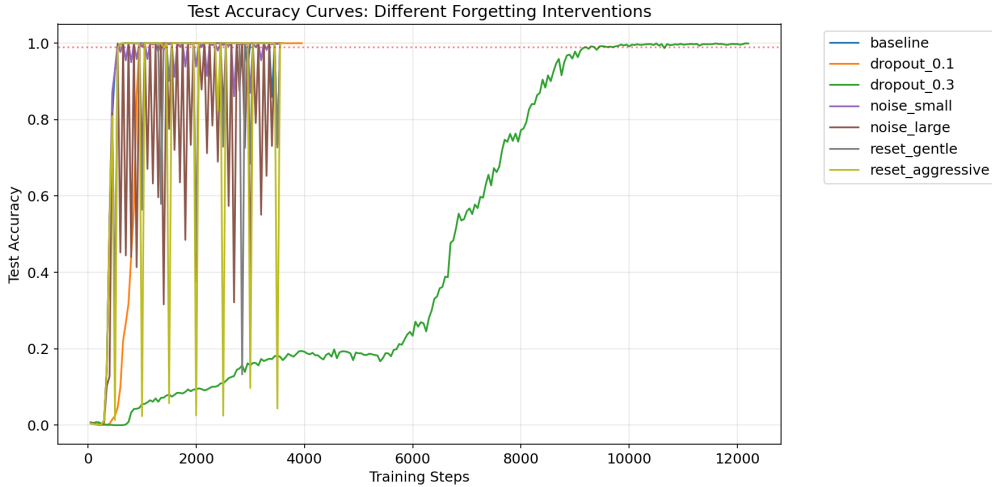


Figure 4: Learning curves for all forgetting interventions. Gentle interventions (reset gentle, noise small) perform comparably to the weight decay baseline, while excessive forgetting (dropout 0.3) catastrophically delays generalization.

et al. [2022] is revealed to be a consequence of insufficient forgetting: when forgetting pressure is strong enough, there is no prolonged memorization phase at all.

Why weight decay works as a forgetting mechanism. Weight decay acts as a continuous pressure toward zero weights. The memorized solution—which stores each training example as a distinct pattern in the weight matrix—requires large, example-specific weights. The generalizable solution—which encodes modular arithmetic as a structured, low-rank computation—uses smaller, more regular weights. Weight decay selectively destabilizes the former while preserving the latter. This makes it a naturally “targeted” forgetting mechanism, unlike dropout, which disrupts representations indiscriminately.

The inverted-U of forgetting. Our Experiment 3 reveals that the relationship between forgetting and generalization is not monotonic for all mechanisms. While weight decay shows a monotonic benefit across our tested range, dropout 0.3 catastrophically delays grokking. The key distinction is whether the forgetting mechanism targets memorized representations specifically or disrupts all representations equally. Effective forgetting must be *selective*: it should destabilize memorized solutions while leaving generalizable structure intact.

5.2 Implications

For training practice. Our results suggest that early training should incorporate strong forgetting pressure (high weight decay), potentially tapering as the model transitions from memorization to generalization. This is consistent with common training schedules that use warmup followed by

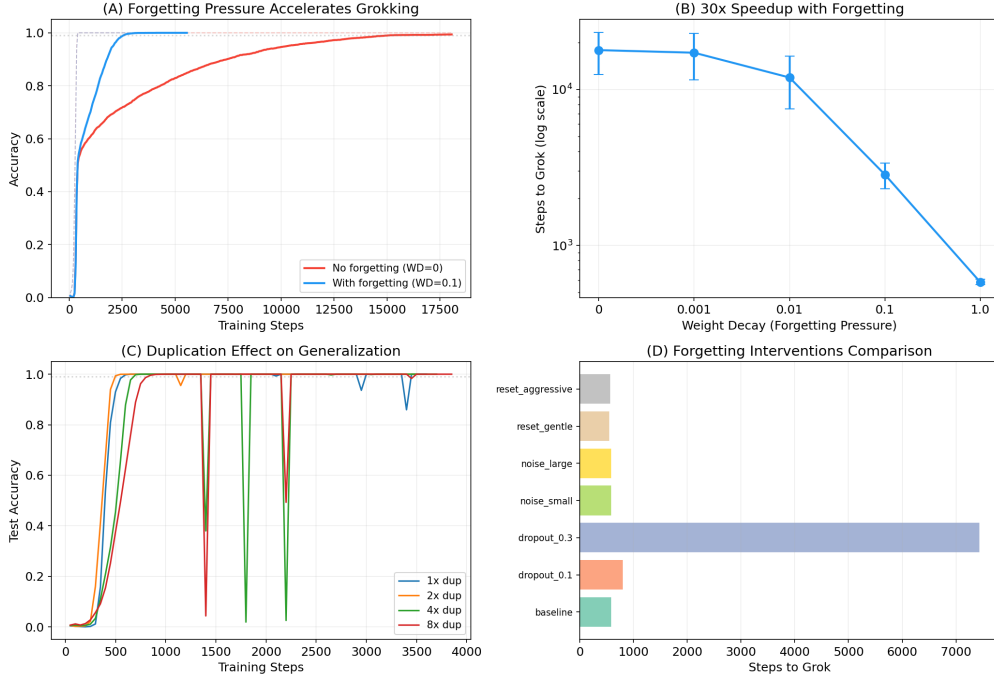


Figure 5: Summary of all three experiments. (a) Weight decay monotonically accelerates grokking (30 $\times$ speedup). (b) Per-example forgetting events increase sharply at high weight decay. (c) Data duplication slows generalization by up to 37%. (d) Forgetting interventions reveal an optimal forgetting level—too little or too much hurts.

cosine decay, though our work suggests the regularization schedule deserves as much attention as the learning rate schedule.

For data curation. The 37% slowdown from 8 $\times$ duplication, even under strong forgetting pressure, reinforces the importance of deduplication Lee et al. [2022], Abbas et al. [2024]. Redundant examples do not help and can actively harm generalization. Combined with the findings of Muennighoff et al. [2023] on diminishing returns from data repetition, this suggests that data diversity is more valuable than data volume.

For the field. Grokking Power et al. [2022], forgetting events Toneva et al. [2019], and memorization-compression cycles Yu [2025] are different views of the same underlying phenomenon: the transition from memorized to generalizable representations. Our work provides the controlled experimental evidence linking these perspectives through the lens of forgetting pressure.

5.3 Limitations

Scale. Our experiments use a 1-layer transformer on a synthetic task with 9,409 examples. While the grokking phenomenon has been observed at LLM scale Li et al. [2025], Yu [2025], we do not directly demonstrate that the 30 $\times$ acceleration from forgetting pressure transfers to models with billions of parameters trained on natural language.

Task complexity. Modular arithmetic has a single, clean generalizable solution. Real-world language tasks involve partial, overlapping, and hierarchical generalizations. The sharp memorize-then-generalize transition may be softer in practice, and the optimal forgetting pressure is likely task-dependent.

Weight decay as forgetting. Weight decay is a regularizer that induces forgetting-like behavior, but it is not the only or necessarily the best mechanism for beneficial forgetting. More sophisti-

cated approaches—such as information bottleneck methods Wang et al. [2025] or forgetting-aware curriculum learning—may be more effective at scale.

Statistical power. We use 3 random seeds per condition. While the observed effects are large and consistent (Cohen’s $d = 3.91$), additional seeds would strengthen confidence in smaller effects, particularly in Experiment 2 where the duplication penalty is more modest.

Confound in Experiment 2. We ran the duplication experiment with $\lambda = 1.0$ (high forgetting pressure), which may have masked the duplication effect. Running at lower weight decay levels would likely reveal a larger penalty from redundant examples, as the model would have less implicit forgetting to compensate.

6 Conclusion

We tested the hypothesis that rapid memorization limits neural network generalization by systematically varying forgetting pressure in the grokking setting. Our central finding is that increasing weight decay from 0 to 1.0 accelerates generalization by $30\times$, with a strong negative correlation between forgetting pressure and time-to-generalization ($\rho = -0.88$, $p < 0.00002$). Models that forget individual examples more readily—as measured by per-example forgetting events—discover generalizable solutions faster. We further showed that data duplication slows generalization and that an optimal forgetting level exists, with weight decay outperforming dropout, noise injection, and weight reset as a forgetting mechanism.

These results reframe the data efficiency problem: rather than asking “how can we give models more data?” we should ask “how can we help models forget what they have memorized?” The gap between human and machine data efficiency may stem not from architectural limitations but from insufficient forgetting pressure during training.

Future work should test whether these findings scale to LLM pretraining, develop principled methods for setting optimal forgetting pressure analogous to Chinchilla scaling laws, and explore forgetting-aware curriculum learning that alternates between memorization and compression phases. The connection between grokking, forgetting events, and compression cycles deserves formalization as a unified theoretical framework.

References

- Amro Abbas, Kushal Tirumala, Daniel Simig, Surya Ganguli, and Ari S. Morcos. SemDeDup: Data-efficient learning at web-scale through semantic deduplication. In *International Conference on Learning Representations*, 2024.
- Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *International Conference on Learning Representations*, 2023.
- Vitaly Feldman and Chiyuan Zhang. Does learning require memorization? A short tale about a long tail. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, 2020.
- Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- Zhenyu Li, Yuyang Fan, and Ziyi Zhou. Grokking in LLM pretraining? Monitor memorization-to-generalization. *arXiv preprint arXiv:2506.21551*, 2025.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Noua-mané Tazi, Sampo Pyysalo, and Thomas Wolf. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 2023.

- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. In *International Conference on Learning Representations*, 2022.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. In *Journal of Machine Learning Research*, volume 15, pages 1929–1958, 2014.
- Kushal Tirumala, Daniel Simig, Surya Ganguli, and Ari S. Morcos. D4: Improving LLM pretraining via document de-duplication and diversification. *arXiv preprint arXiv:2308.12284*, 2023.
- Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2019.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Zhe Wang et al. Breaking memorization barriers in LLM code fine-tuning via information bottleneck. *arXiv preprint arXiv:2510.16022*, 2025.
- Zhichao Yu. Memorization-compression cycles improve generalization. In *NeurIPS Workshop*, 2025.
- Chiyuan Zhang, Daphne Ippolito, Katherine Lee, Matthew Jagielski, Florian Tramer, and Nicholas Carlini. Counterfactual memorization in neural language models. In *International Conference on Learning Representations*, 2024.
- Xuanming Zheng et al. Spurious forgetting in continual learning of language models. In *International Conference on Learning Representations*, 2025.